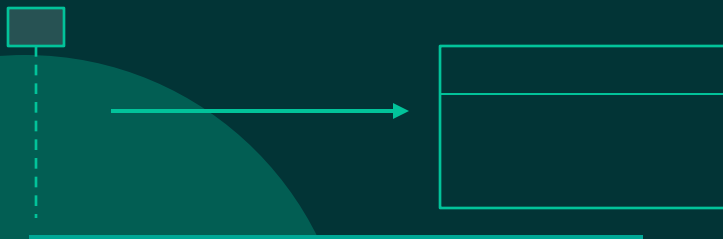


UML STRUCTURAL & BEHAVIORAL MODELING

Sequence & Class Diagrams

Introduction, Notation & Rules

- *Dr Madhavi Dave*



Interaction & Structural / UML Modeling

What Is a Sequence Diagram — and Why Use One?

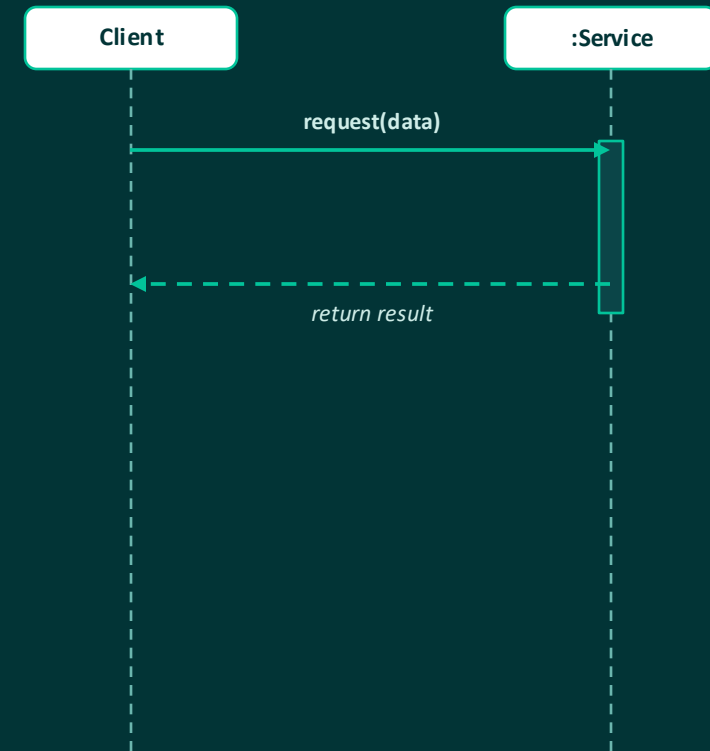
A **sequence diagram** is a UML interaction diagram that shows how a set of objects **collaborate over time** — the exact order of messages exchanged to carry out one scenario.

Pressman groups it with behavioral models built during requirements analysis: where the activity diagram shows the **logical flow** of a process, the sequence diagram adds the missing dimension — **which object sends which message to which other object, and in what order**.

Why it matters

- Validates that a use case is actually realizable by the objects in the design.
- Surfaces the operations (methods) each class must expose, before coding begins.
- Distinguishes synchronous calls (caller waits) from asynchronous ones (caller continues).
- Makes timing and ordering assumptions explicit and reviewable.

AT A GLANCE



Messages read top-to-bottom in strict time order — the vertical axis is time, not layout.

Core Symbols Used in a Sequence Diagram



Lifeline

A dashed vertical line dropping from an object or actor header; represents its existence through time.



Activation Bar

A thin rectangle on a lifeline marking the span during which that object is actively executing.



Synchronous Message

Solid arrow, filled triangular head — the caller blocks and waits for the call to return.



Return Message

Dashed arrow, open head — shows a value passed back to the caller; closes an activation.



Combined Fragment (alt / loop)

A labeled rectangular frame enclosing messages that are conditional or repeated.

Rules for Constructing a Sequence Diagram

1

Time Flows Downward

The vertical axis always represents elapsed time; never reorder messages for layout convenience.

2

Order Lifelines by Role

Place the initiating actor at the far left, followed by collaborators in the order they are first contacted.

3

One Activation per Call

Every activation bar corresponds to exactly one method execution on that lifeline.

4

Every Sync Call Returns

Close a synchronous call with a dashed return arrow before its activation bar ends.

5

Label with Signature

Each message name should read like an operation call: verb plus key parameters.

6

Guard Conditions Are Explicit

Alt and opt frames state their branching condition in brackets in the frame's tag.

7

Bound Repetition with Loop

Wrap repeated message sequences in a loop frame labeled with its iteration condition.

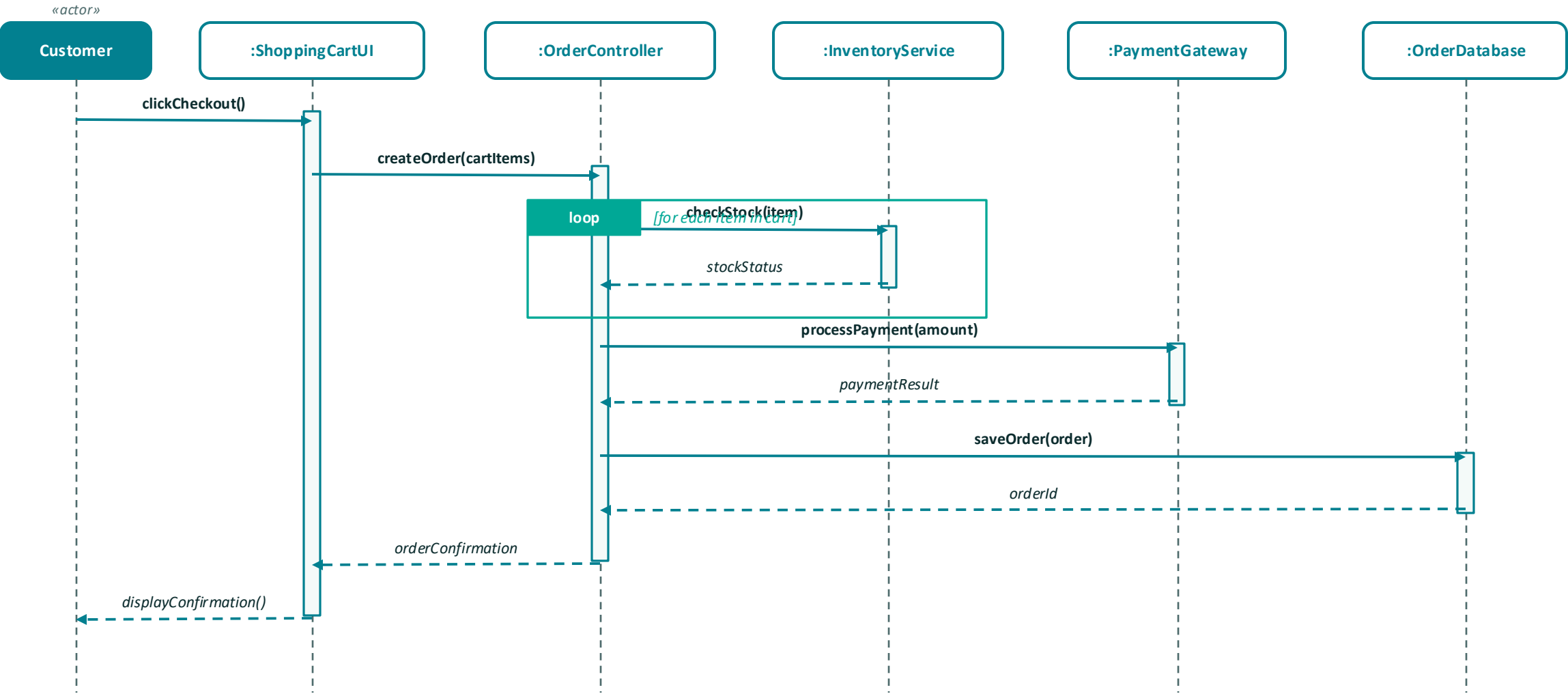
8

One Scenario per Diagram

Model a single path through a use case; branch-heavy logic belongs in an alt fragment or a separate diagram.

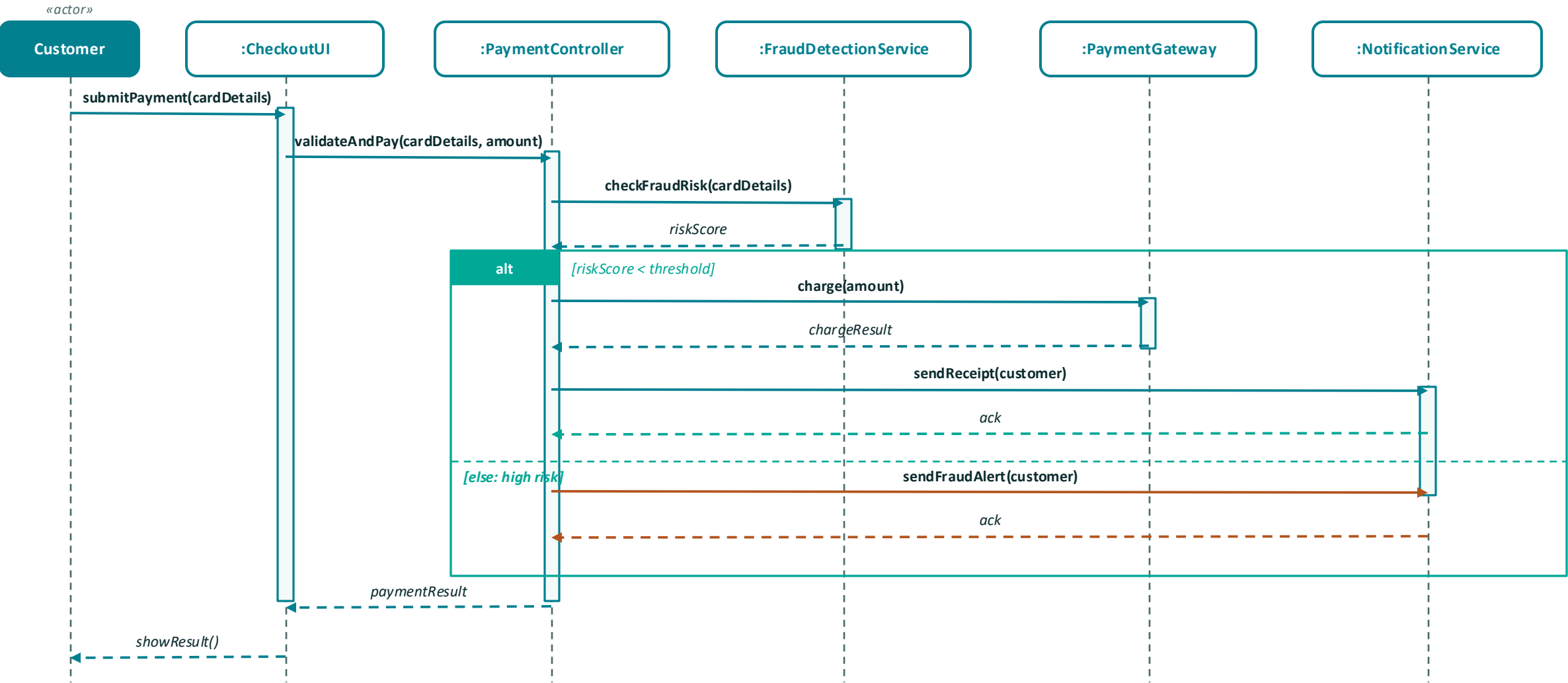
Sequence Diagram — Place Order (Online Shopping)

Traces checkout from the customer's click through stock verification (looped per cart item), payment, and order persistence.



Sequence Diagram — Payment Processing with Fraud Check

A more branching scenario: a risk score determines whether the diagram's alt fragment charges the card or raises a fraud alert.



What Is a Class Diagram — and Why Use One?

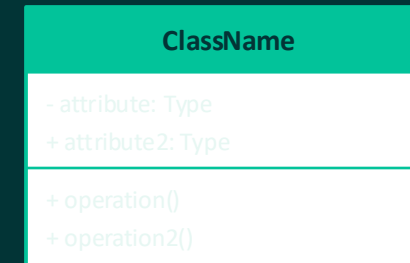
A **class diagram** is a UML structural diagram that models the **static shape** of a system: the classes it is built from, the attributes and operations each one owns, and how the classes relate to one another.

Pressman places class diagrams inside **class-based modeling**: candidate classes are identified by a grammatical parse of the problem statement and use cases — the **nouns** become classes, the **verbs** become operations.

Why it matters

- Establishes a shared vocabulary — the domain model — for analysts, designers, and developers.
- Specifies each class's responsibilities: what it knows (attributes) and what it does (operations).
- Makes structural relationships and their multiplicities explicit before coding starts.

AT A GLANCE



visibility: +public -private #protected

Relationships travel between class boxes as lines — plain for association, diamonds for whole/part, triangles for is-a.

Core Symbols Used in a Class Diagram

**Class Box**

Three compartments: class name, attributes, and operations — each attribute/operation prefixed with visibility.

**Association**

A plain line between two classes; each end may carry a role name and a multiplicity (1, 0..1, 0..*, 1..*).

**Aggregation**

A line with a hollow diamond at the “whole” end — a has-a relationship where parts can outlive the whole.

**Composition**

A line with a filled diamond at the “whole” end — parts cannot exist once the whole is destroyed.

**Generalization / Realization**

A line ending in a hollow triangle points to a superclass (solid line) or an interface (dashed line).

Rules for Constructing a Class Diagram

1

Mine Nouns for Classes

Start from the problem statement and use cases; candidate classes surface as the significant nouns.

2

One Class, One Responsibility

Each class should represent a single, cohesive concept — split classes that are doing two jobs.

3

State vs. Behavior

Attributes capture what an object knows; operations capture what it can do. Keep both minimal.

4

Mark Visibility Correctly

Default attributes to private (–); expose only the operations (+) that collaborators actually need.

5

Pick the Right Relationship

Association for “uses,” aggregation for a separable has-a, composition for a lifecycle-bound has-a, generalization for is-a.

6

Always State Multiplicity

Every association end declares how many instances participate: 1, 0..1, 0..*, or 1..*.

7

Don't Duplicate Inheritance

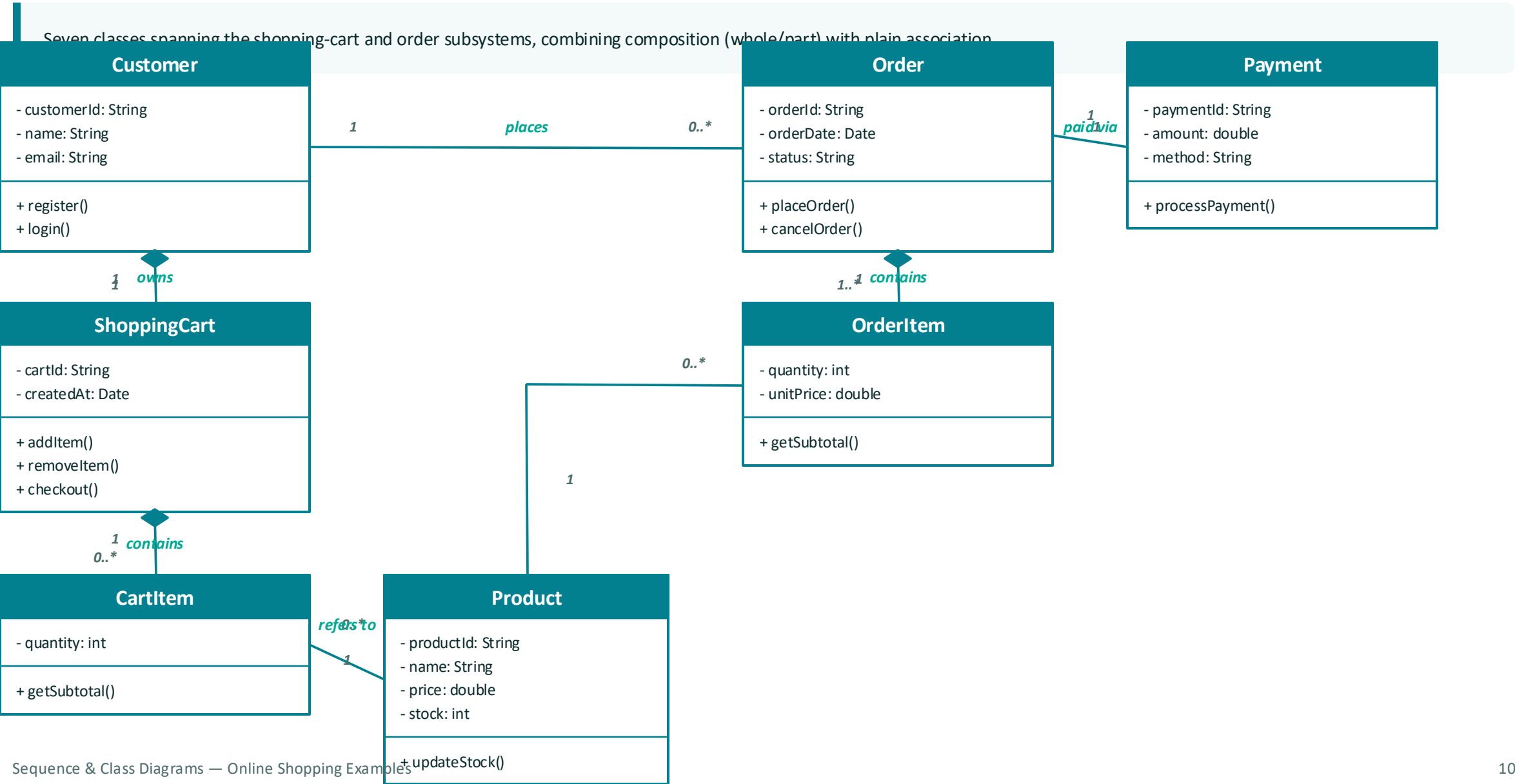
If a relationship is already implied by a superclass, don't redraw it on every subclass.

8

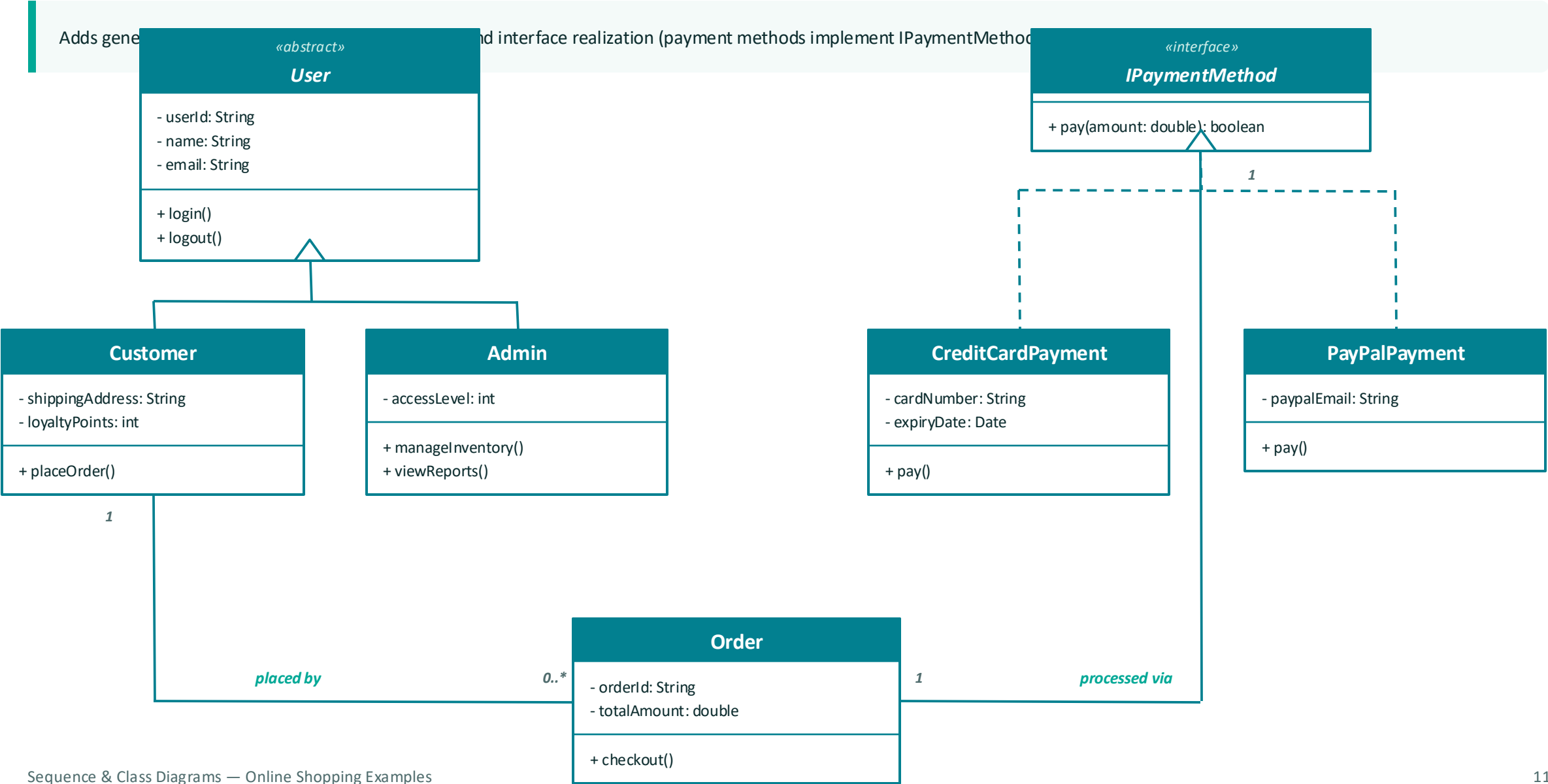
Keep One Level of Abstraction

Don't mix analysis-level domain classes with implementation-only utility or helper classes.

Class Diagram — Online Shopping Domain Model



Class Diagram — User Roles & Payment-Method Hierarchy



SUMMARY

Key Takeaways

- 1 Sequence diagrams capture interaction over time — which object messages which other object, and in what order — validating that a use case is realizable.
- 2 Class diagrams capture structure — the classes, attributes, operations, and relationships that make up the static domain model.
- 3 Consistent notation (lifelines, activations, messages, frames for sequence; boxes, associations, diamonds, triangles for class) keeps both readable across teams.
- 4 Construction rules for each diagram type ensure scope stays disciplined: one scenario per sequence diagram, one abstraction level per class diagram.
- 5 The online-shopping examples show real complexity: looped and branching interactions, plus composition, association, generalization, and interface realization together.